

BTP 1st review of given research paper

Joel James & Majida Bee
24 August 2026

iRDRC: An Intelligent Real-Time Dual-Functional Radar-Communication System for Automotive Vehicles

What is this paper trying to solve?

An autonomous vehicle needs two capabilities that compete for the same resources. First, it must sense its surroundings and detect unexpected objects or events that could cause a collision. Second, it must communicate information such as traffic data, sensor data, or live video to roadside infrastructure and base stations. The paper combines these two jobs in one Dual-Functional Radar-Communication (DFRC) system.

The important point is that radar and communication do not run together at every time step. The vehicle must decide which one deserves the shared resource right now. If it uses radar too often, fewer packets are transmitted. If it communicates too often, an unexpected event may go undetected. Therefore, the central problem is a safety-versus-throughput decision under an uncertain and changing environment.

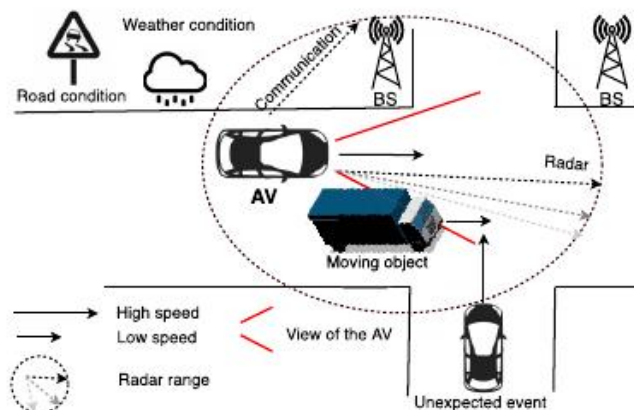


Fig. 1. Autonomous vehicle with a DFRC system.

Earlier approaches divide time between radar and communication using fixed schedules. For example, an IEEE 802.11ad-based approach reserves certain frame blocks for radar and others for data. Another approach divides a time cycle into fixed radar and communication portions. These methods are simple, but they assume that the environment is relatively predictable.

Figure 1 from the paper shows the whole picture at a glance. The car (AV, for “Autonomous Vehicle”) sits at the center. Around it are the four things that make the road risky - weather, road condition, speed, and nearby moving objects - plus two possible uses of its single radio: sending data up to Base Stations (the communication mode) or scanning outward for an “unexpected event”, such as a hidden vehicle about to pull out (the radar mode).

System Model: What information does the AV use?

At every time step, the vehicle chooses exactly one mode. The communication side has a queue of data packets, while the radar side looks for unexpected events outside the normal camera view or hidden by other objects/weather.

Communication mode ($a = 0$): send some of the data waiting in its queue to a BS.

- New packets arrive at the queue (modeled as Poisson with rate $\lambda_d = 1$ packet/step).
- If the queue is full ($d = D$), new arrivals are dropped.
- If the AV chooses communication mode ($a = 0$):
 - It transmits some packets over the channel.
 - How many packets get through depends on the channel state $c \in \{0, 1\}$:
 - * $c = 0$ (good channel): transmit $v_1 = 4$ packets successfully.
 - * $c = 1$ (bad channel, high interference): transmit $v_2 = 2$ packets successfully.
 - The channel switches between good/bad randomly each step, with $P(\text{bad channel}) = p_c = 0.1$ (i.e., 90% of the time the channel is good).

Queue dynamics: $d_{t+1} = \min(d_t - (\text{transmission count if } a = 0) + (\text{arrivals}), D)$.

The throughput metric is the average number of packets successfully transmitted per time step over a long episode.

Radar mode ($a = 1$): scan the surroundings for a hidden or unexpected obstacle.

When radar mode is selected, the AV performs automotive millimeter-wave radar sensing. An unexpected event is something that can potentially cause a collision and is outside the effective camera view - for example, a vehicle emerging from another road while hidden behind a truck. The original paper assumes the radar itself is perfect: if an event exists and radar mode is selected, it detects the event.

The paper makes a simplifying assumption: the radar is treated as perfect. If there really is a hidden obstacle out there, radar mode will always catch it. Nothing gets missed when radar is switched on. The only way an obstacle goes undetected is if the car happened to be in communication mode at that exact moment and an unexpected event occurred.

Factor	Variable	State 0: Favorable	State 1: Unfavorable	Why it matters
Road condition	r	Straight / good surface	Slippery / wet	Less grip can increase collision risk.
Weather	w	Clear	Rain / poor weather	Poor visibility makes hazards harder to see.
AV speed	v	Low	High	Higher speed leaves less reaction time.
Moving object	m	No nearby object	Nearby moving object	Moving traffic can create sudden hazards.

The two performance measures:

- Data throughput: average number of packets successfully transmitted per time unit.
- Miss-detection probability: fraction of unexpected events that the AV fails to detect.

These measures pull the decision in opposite directions. More radar generally improves detection safety but reduces the time available for communication. More communication improves throughput but can increase the chance that an unexpected event occurs while radar is not active.

Environment Model and Event Probability Eq 1:

The paper needs a way to estimate how likely an unexpected event is under the current environmental conditions. It defines, for each factor i , two conditional probabilities: p_0^i for the favorable state and p_1^i for the unfavorable state. It also defines τ_i as the probability that factor i is in the favorable state (0). Therefore, the probability of the unfavorable state is $1 - \tau_i$. The paper combines the four factors using the following expression:

$$P(\oplus) = \sum_{i \in \{r,w,v,m\}} [\tau_i p_0^i + (1 - \tau_i) p_1^i] \quad (1)$$

For each factor, first calculate a weighted event probability using its favorable-state probability and unfavorable-state probability. Then multiply the four factor terms together. The model assumes the factors are independent.

Parameter	Value	Meaning
p_0^v	0.005	Event probability under low speed.
p_1^v	0.1	Event probability under high speed.
p_0^w	0.005	Event probability under favorable weather.
p_1^w	0.046	Event probability under unfavorable/rainy weather.
P_0^r, P_1^r	Not specified	Road-condition probabilities are assumed/pre-defined in the model.
P_0^m, P_1^m	Not specified	Moving-object probabilities are assumed/pre-defined in the model.

Suppose high speed becomes much more likely to produce an unexpected event. The term involving p_1^v becomes more important, so the overall risk estimate rises. The learning system does not simply react to speed alone; it sees speed together with road, weather, moving-object, queue and channel information. This is what allows the policy to become context-dependent.

The product form assumes the factors are independent. In a real vehicle, this is not always true. Rain can make the road slippery, and bad weather may also change traffic behavior. Therefore, a more realistic future model could learn a joint distribution between road, weather, speed and nearby objects instead of treating them as independent. This is one of the clearest modeling limitations identified during the reading of the paper.

Turning the problem into a Markov Decision Process (MDP):

An MDP gives a clean mathematical structure for repeated decision-making: the vehicle observes a state, chooses an action, receives a reward and moves to a new state. The important practical issue is that the state-transition probabilities are not assumed to be known by the AV.

State Space Eq 2:

$$S = \{ (d, c, r, w, v, m) : d \in \{0, \dots, D\}, c \in \{0, 1\}, r, w, v, m \in \{0, 1\} \} \quad (2)$$

The six parts of the state are:

Symbol	What the AV observes	Values	Decision relevance
d	Number of packets waiting in queue	0 to D; D=10 in simulation	High queue → communication may be valuable.
c	Communication-channel condition	0 good / 1 bad	Good channel allows more packets to be sent.
r	Road condition	0 / 1	Unfavorable road increases safety risk.
w	Weather	0 / 1	Bad weather can increase unexpected-event risk.
v	AV speed	0 / 1	High speed reduces reaction time.
m	Nearby moving object	0 / 1	Nearby traffic can create sudden hazards.

With $D = 10$, the number of possible states is $11 \times 2 \times 2 \times 2 \times 2 \times 2 = 352$. This is not enormous, but it is already large enough to motivate a function-approximation method rather than maintaining a simple table for every situation.

Action Space:

$A = \{0, 1\}$ where $a = 0$ means communication mode and $a = 1$ means radar mode. This is the central decision of the entire system. The AV is not choosing how much power to allocate or how to shape a radar waveform. It is deciding which function should use the shared DFRC resource at this time step.

Reward Function Eq 3:

The reward is the bridge between the engineering goals and the reinforcement-learning algorithm. Instead of directly telling the agent “maximize throughput and safety,” the paper assigns numerical rewards and penalties that represent those goals.

Action	Situation	Reward	Meaning
Communication	Good channel, no event	$+r_1 = +2$	Successful high-rate communication.
Communication	Bad channel, no event	$+r_2 = +1$	Communication succeeds, but less data is sent.
Communication	Unexpected event occurs	$-r_3 = -50$	Large safety penalty for not using radar.
Radar	No event	0	Radar was used but found nothing.
Radar	Event occurs	$+r_4(b+1) = +5(b+1)$	Successful detection; reward increases with risk.

Here, b is the number of unfavorable conditions among road, weather, speed and moving-object states. If $b = 4$, then a detected event gives $5(4+1) = +25$. The strongest signal is the -50 penalty

for communicating when an unexpected event occurs. This intentionally creates a safety-first reward structure.

Objective and the Deep Q-Network Eq 4:

A reinforcement-learning agent should not only chase the best immediate reward. The AV needs a policy that remains useful over many future time steps. The paper therefore maximizes the expected discounted return:

$$\max_{\pi} G(\pi) = E \{ \sum_{t=0}^T \gamma^t r_{t+1}(\pi) \} \quad (4)$$

Here, π is the policy that maps the observed state to an action; T is the time horizon; and $\gamma \in (0,1)$ is the discount factor. A larger γ gives more importance to future rewards. The optimal policy π^* is the policy that maximizes this expected cumulative reward.

- $r_{t+1}(\pi)$ = immediate reward received at time step $t+1$ under policy π .
- $\gamma \in (0,1)$ = discount factor (paper doesn't specify; typical value ≈ 0.99).
 - γ close to 1 means future rewards matter almost as much as immediate rewards.
 - γ close to 0 means the AV is myopic, only caring about immediate rewards.
- T = time horizon (episode length; paper doesn't specify exactly, but simulation episodes appear to run for hundreds of steps).

The optimal policy π^* maximizes this expected return:

$$\pi^* = \arg \max_{\pi} G(\pi)$$

Intuitively: the optimal policy is the one that, when followed, leads to the highest expected cumulative reward over the long run. It balances immediate gains (transmitting packets) against long-term risks (missing obstacles).

Why not ordinary Q-Learning?

Classical Q-learning estimates a Q-value $Q(s,a)$ for every state-action pair and stores these values in a table. In this problem the state has 352 possible combinations, and a larger real-world model could have far more. A table can become inconvenient as the state space grows.

The paper therefore uses Deep Q-Network (DQN). Instead of a Q-table, a neural network receives the current state and outputs the estimated Q-values for the possible actions. The action with the better learned value can then be selected, while ϵ -greedy exploration allows the vehicle to keep trying alternatives during training.

DQN Learning Process:

Step	Operation	What happens
1	Observe state	The AV observes d, c, r, w, v and m.
2	Estimate Q-values	The neural network estimates $Q(s,a,\theta)$ for communication and radar.
3	Choose action	An ϵ -greedy policy balances exploration and exploitation.
4	Interact	The AV receives reward r_t and observes the next state s_{t+1} .
5	Store experience	The transition (s_t, a_t, r_t, s_{t+1}) is stored in replay memory.
6	Train	A random mini-batch is sampled from memory and used to update the network.
7	Repeat	The process continues over episodes until cumulative reward becomes stable.

Network Update Eq 5:

$$\theta_{t+1} = \theta_t + \alpha [y_t - Q(s_t, a_t; \theta_t)] \nabla Q(s_t, a_t, \theta_t) \quad (5)$$

The term y_t is the target used for learning: $y_t = r_t + \gamma \max_a Q(s_{t+1}, a; \theta^-_t)$. The online network has weights θ_t , while θ^-_t represents a target network that is copied periodically from the online network. The difference between the target and the current estimate tells the network how it should adjust its weights.

Why replay memory matters: consecutive vehicle experiences are strongly related. Randomly sampling stored transitions creates more varied training batches and makes learning more stable. The target network also reduces the problem of the target changing too quickly during training.

Experimental Setup and Results:

The paper evaluates the proposed DQN against two baselines: tabular Q-learning and a simple Round-robin policy. Round-robin alternates between radar and communication without considering the current environment. The comparison asks three main questions: Does the method learn? Does it converge quickly? Does it improve throughput and safety compared with fixed scheduling?

Main simulation settings

Parameter	Setting	Role
Queue capacity D	10 packets	Maximum stored data.
Packet arrival rate λ_d	1 packet/step	Average incoming data.
Good-channel transmission v_1	4 packets	Successful packets in good channel.
Bad-channel transmission v_2	2 packets	Successful packets in bad channel.
Bad-channel probability p_c	0.1	Channel is good 90% of the time.
Rewards (r_1, r_2, r_3, r_4)	(2, 1, 50, 5)	Strongly penalizes missed safety events.
High-speed event probability p_1^v	0.1 $\rightarrow$ 1.0 sweep	Used to study response to increasing risk.
DQN network	MLP: 6 inputs, 2 hidden layers, 2 outputs	Maps state to two action Q-values.
Implementation	Keras + OpenAI Gym	DQN and AV environment.

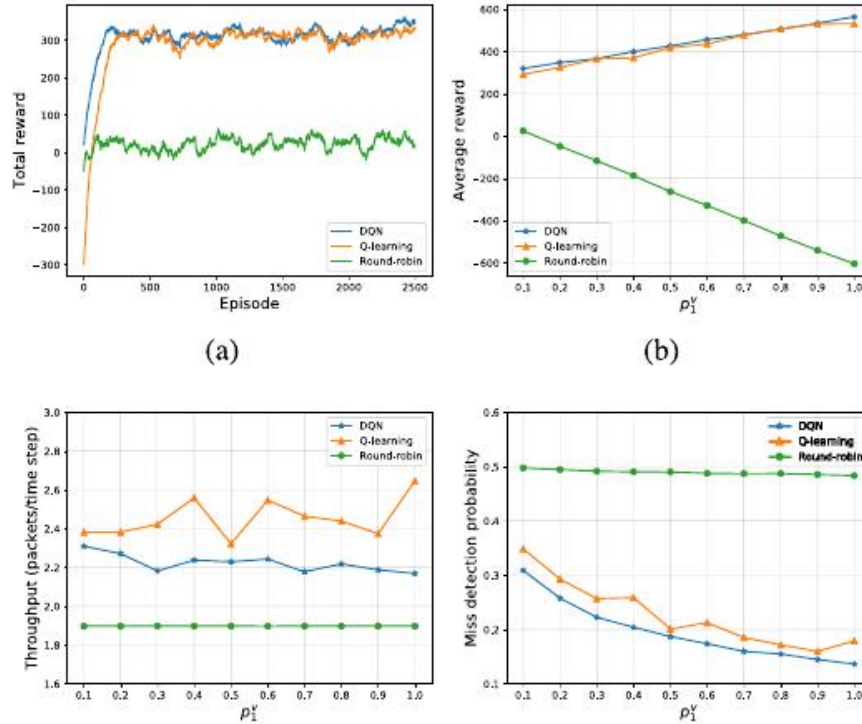


Figure 2. Original paper results: (a) total reward versus episode, (b) average reward versus high-speed event probability, (c) throughput, and (d) miss-detection probability.

Result (a): learning and convergence:

DQN and Q-learning reach much higher total rewards than Round-robin. DQN and Q-learning eventually reach similar reward levels, but DQN learns faster. The paper reports that DQN approaches the optimal value in about 170 episodes, while Q-learning takes about 280 episodes. The main reason is that DQN updates from mini-batches of stored experiences, whereas tabular Q-learning updates one Q-value at a time.

Result (b): what happens when risk increases?

As p_1^v increases, Round-robin's average reward falls because its fixed schedule can continue using communication even when the chance of an unexpected event becomes high. DQN and Q-learning improve because their learned policies respond by selecting radar more often when the environment becomes riskier.

Adapting to danger: as the chance of a high-speed event (p_1^v) climbs from 0.1 to 1.0, the Round-robin scheme's average reward collapses (it keeps missing events because its schedule is fixed), while DQN and Q-learning's rewards climb, proof that they are correctly switching to radar mode more often exactly when danger increases.

Result (c): throughput :

The throughput plot shows that DQN and Q-learning maintain better packet transmission performance than Round-robin under the tested conditions. This is important because the intelligent policy is not simply "always choose radar." It chooses radar when safety signals justify it and communication when transmission is more useful. That is the heart of the adaptive strategy.

Result (d): miss-detection probability :

The DQN and Q-learning miss-detection probabilities decrease as p_1^v increases. This may seem counter-intuitive at first: the environment becomes more dangerous, yet the measured miss-detection probability becomes lower. The reason is that the learned policy reacts to the increased risk by selecting radar more frequently. Round-robin stays near a much higher miss-detection level because it does not adapt to the changing risk.

The paper reports a miss-detection probability roughly in the 0.15–0.30 range under the stated reward setting (2, 1, 50, 5). It also shows that increasing r_4 — the reward for successful radar detection — can push the policy toward more radar use and further reduce miss detection. This demonstrates that the reward design is not a cosmetic choice; it directly affects the safety-throughput balance.

Takeaways :

Stepping back, the paper makes three contributions worth remembering:

- (1) it frames the age-old radar-vs-communication scheduling problem as a Markov Decision Process instead of a fixed timetable
- (2) it designs a reward function that bakes in a strong safety-first bias (a huge penalty for missed hazards, modest rewards for data) and
- (3) it shows that a Deep Q-Network can learn the right balance on its own, faster than plain Q-learning and far better than naive scheduling.

- The radar is assumed to be perfectly accurate - real millimeter-wave radar can still miss things or throw false alarms, especially in heavy rain, which the authors themselves flag as a straightforward future extension.
- Only two of the four risk-factor probabilities (speed and weather) are grounded in real-world data; road-condition and moving-object probabilities are left unspecified, so anyone reproducing the results has to choose reasonable stand-in values.
- The model assumes the four risk factors are independent of one another, when in reality bad weather and slippery roads, for instance, tend to go hand in hand.
- Only a single vehicle is considered - the paper's own conclusion points to cooperation between multiple connected vehicles as an interesting next step.

Modeling Gaps and Questions for Implementation :

As noted above, several aspects of the system model are underspecified in the paper and will need explicit choices during implementation:

1. Missing probability values: P_{0^r} , P_{1^r} , P_{0^m} , P_{1^m} are not given. Reasonable estimates will be chosen and documented.
2. State evolution: How do r, w, v, m evolve over time? The paper suggests each is resampled per step, but τ_i (state-transition probabilities) are not given. Again, reasonable defaults will be chosen.
3. Exact event-generation process: Equation 1 gives $P(\oplus)$ as a marginal probability. The paper doesn't explicitly state whether each step's event ($\oplus$ or $\neg\oplus$) is a Bernoulli draw with this probability. We'll assume yes, but this is an assumption.
4. Discount factor γ and episode length T : Not specified in the paper. Typical values will be used ($\gamma \approx 0.99$, $T \approx 500 - 1000$ steps per episode).
5. DQN hyperparameters: learning rate, batch size, replay buffer size, network architecture, ϵ decay schedule - none of these are fully specified in the paper's 4-page format. Classical DQN choices will be documented.

Complete Paper in one mental model :

Environment $\rightarrow$ road + weather + speed + moving object + queue + channel

Risk model $\rightarrow$ Equation (1) estimates unexpected-event likelihood

MDP $\rightarrow$ Equation (2) defines the state and actions define radar/communication

Reward $\rightarrow$ Equation (3) converts throughput and safety into learning signals

Objective $\rightarrow$ Equation (4) maximizes long-term discounted reward

DQN $\rightarrow$ Equation (5) learns Q-values from experience

Policy $\rightarrow$ choose communication when it is useful, radar when safety risk is high

Outcome $\rightarrow$ higher reward, faster convergence, better throughput and lower miss detection than fixed Round-robin in the simulations.